OrchAI

GitHub Araştırma Raporu

LLM Orchestration / Memory / RAG / Eval / Tracing — Güncel Yaklaşımlar ve Öneriler

Mart 2026

---

1. LLM Orchestration

LLM orkestrasyon katmanı; birden fazla LLM çağrısı, arac kullanimi ve karar noktaları iceren iş akışlarını koordine eden modüldür. Asagida en yaygın yaklaşımlar karşılaştırılmıştır.

Yaklaşım	Artıları	Eksileri
LangChain	Geniş ekosistem, hazır entegrasyonlar, LCEL zinciri	Aşırı soyutlama, debug zorluğu, versiyon kırılımları
LlamaIndex	RAG odakli, veri bağlayıcıları güçlü, index çeşitliliği	LLM orchestration sınırlı, RAG dışında zayıf
Custom Orchestrator (OrchAI)	Tam kontrol, minimal bağımlılık, test kolaylığı	Başlangıç maliyeti yüksek, ekosistem yok
AutoGen (Microsoft)	Çok-ajan senaryoları, ajan diyalogu güçlü	Production olgunluğu düşük, karmaşık debug
Haystack	Production odaklı, pipeline tabanlı, tip güvenliği	Öğrenme eğrisi dik, RAG dışında az kullanım

> *Oneri: OrchAI use-case'i için Custom Orchestrator doğru seçimdir. Sistemin routing mantığı basit ve öngörülü, LangChain gibi ağır bir çerçeve gereksiz karmaşıklık ekler. Mevcut mimari ile tam kontrol, kolay test ve minimal bağımlılık sağlanmıştır.*

---

2. Memory / Context Yonetimi

LLM uygulamalarinda hafıza yönetimi; konusma bütunlugu, kullanıcı tercihleri ve uzun vadeli bilgi saklamak için kritiktir.

Yaklaşım	Artıları	Eksileri
Rolling Window (Son N mesaj)	Basit, öngörülü, düşük maliyet	Eski bilgi kaybolur, N büyürse maliyet artar
Summarization Memory (OrchAI)	Geçmişi korur, token verimli, MongoDB'de kalıcı	Özet kalitesi LLM'e bağımlı, gecikme ekler

Vector Memory (Embedding)	Semantik arama, çok büyük geçmişler için ideal	Karmaşık altyapı, embedding maliyeti
Entity Memory	Varlıkları (kişi, yer, nesne) ayrı izler	Extraction hatalı olabilir, kapsam sınırlı
MemGPT / External Store	Çok büyük context, kalıcı kullanıcı profili	Çok karmaşık, latency yüksek, production zor

> *Oneri: OrchAI'nin Rolling Window + Summarization kombinasyonu bu use-case için optimum dengedir. Kısa vadeli mesajlar ham halde, uzun vadeli geçmiş özet olarak saklanır. Gelecekte hacim büyürse Vector Memory eklenebilir.*

---

3. RAG (Retrieval-Augmented Generation)

RAG; LLM'in bilgi tabanına erişimini sağlar. Farklı RAG yaklaşımlarının karşılaştırması aşağıdadır.

Yaklaşım	Artıları	Eksileri
Naive RAG (OrchAI)	Hızlı, basit, az bağımlılık, anında üretim	Precision sınırlı, re-ranking yok
Advanced RAG (HyDE, re-rank)	Çok daha yüksek precision, sorgu genişleme	Maliyet artar, gecikme artar, karmaşık
Modular RAG	Her adım değiştirilebilir, esnek mimari	Başlangıç yatırımı yüksek
GraphRAG (Microsoft)	İlişkisel bilgi, çok-hop sorgular	Çok karmaşık, küçük veri setlerinde overkill
ColBERT / Dense Retrieval	Yüksek kalite retrieval, çok dilli destek	GPU gerektirebilir, hosting maliyeti

> *Oneri: Mevcut Naive RAG (text search + cosine similarity) PoC için yeterlidir. Precision artırmak için bir sonraki adımda cross-encoder re-ranking veya HyDE (Hypothetical Document Embedding) eklenebilir.*

---

4. Evaluation (Degerlendirme)

LLM çıktılarının kalitesini ölçmek; halüsinasyon, yanıt kalitesi ve tutarlılık açısından kritik önem taşır.

Yaklaşım	Artıları	Eksileri
LLM-as-Judge (GPT-4)	Esnek, doğa dili anlayışı, hakem olarak güçlü	Maliyet, kendi biası, non-deterministic
RAGAS Framework	RAG'a özel metrikler (faithfulness, context recall)	Sadece RAG için, kurulum gerekli
Human Eval	En güvenilir, alana özgü değerlendirme	Yavás, pahali, olceklenmez
Reference-based (ROUGE, BLEU)	Hızlı, deterministik, ücretsiz	Anlambilimsel kaliteyi ölçemez
Offline Test Seti (OrchAI)	Basit, kontrol edilebilir, CI/CD entegre	Manuel güncelleme gerektirir, sınırlı kapsam

> *Oneri: Kısa vadede: /chat/test-scenarios ile offline eval seti yeterli. Orta vadede RAGAS ile RAG kalitesi ölçülebilir. Uzun vadede LLM-as-Judge ile otomatik kalite skoru eklenebilir.*

---

5. Observability ve Tracing

LLM uygulamalarinda her istekte; prompt, model, token, gecikme, araç çağrısı ve maliyet izlenmelidir.

Yaklaşım	Artıları	Eksileri
Custom Mongo Trace (OrchAI)	Tam kontrol, sıfır ek bağımlılık, esnek şema	Görselleştirilmiş dashboard yok, manuel sorgu
LangSmith	LangChain ile entegre, zengin UI, trace görselleştirme	LangChain'e bağımlı, SaaS maliyet
Langfuse (Open-source)	Self-host edilebilir, LLM agnostik, dashboard	Kurulum gerekli, ekstra altyapı
Arize Phoenix	ML+LLM monitoring, embedding drift analizi	Karmaşık kurulum, büyük ekipler için
OpenTelemetry + Grafana	Standart, tüm stack ile entegre, çok güçlü	Dik öğrenme eğrisi, LLM özelleşirmesi gerekir

> *Oneri: Mevcut MongoDB trace_logs yaklaşımı PoC için idealdir. Production'a geçildiğinde Langfuse (open-source, self-hosted) önerilir: LLM agnostik, zengin dashboard, kurulum kolay ve OrchAI'nin mevcut trace şemasıyla uyumlu.*

---

6. Özet Öneri Tablosu

Başlık	Mevcut Yaklaşım	Öneri (Sonraki Adim)
Orchestration	Custom LLMOrchestrator	Mevcut yapı yeterli, agent loop eklenebilir
Memory	Rolling Window + Summarization	Hacim büyürse Vector Memory ekle
RAG	Text Search + Cosine Similarity	Cross-encoder re-ranking ile precision artır
Evaluation	Offline 10-senaryo test seti	RAGAS ile RAG kalitesini ölç
Tracing	MongoDB trace_logs	Production'da Langfuse'e geç
Web Arama	DuckDuckGo Instant API	Serper veya Tavily API ile güçlendir